



TicTacToe

Assessing the Performance of a Minimax AI Agent

Minerva University

Prof. Rohan Shekhar.

CS152 -2nd Semester

Final Project

Problem Definition:

The issue at hand is developing an AI player for a 4x4 Tic Tac Toe game versus a computer-generated adversary and a human player. Tic Tac Toe is a two-player game where players alternately place their respective symbols (X or O) on a square grid in an effort to line up their symbols in a line that is either horizontal, vertical, or diagonal. When a player successfully forms a line, the game is over; otherwise, it is a draw. The game also finishes when no more moves can be made. Finding the best moves to play is a challenge that arises frequently in games, which makes the topic intriguing. AI approaches are a good fit for this problem because they can help the agent explore the search space and make decisions effectively.

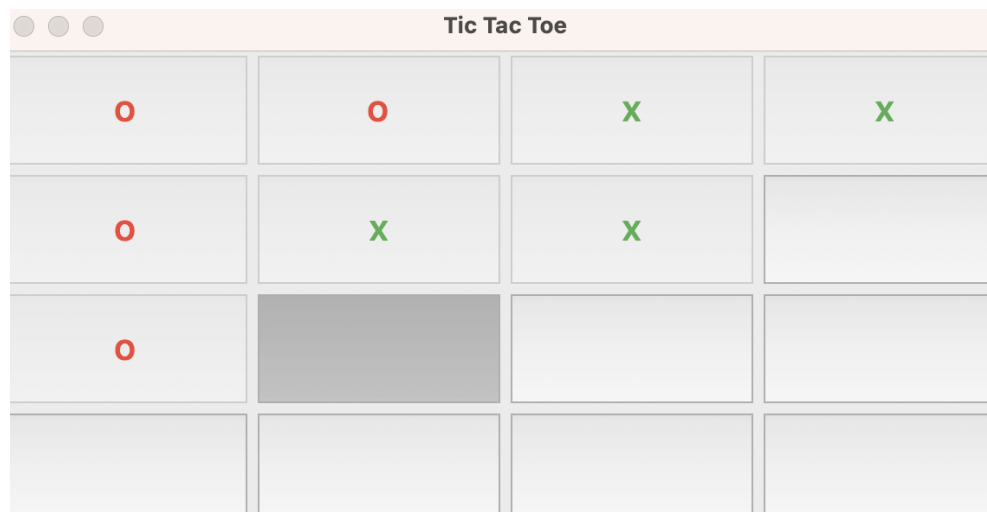


Figure 1. An illustration of the 4 * 4 tic-tac-toe game.

Solution Specification:

To address the issue, our code defines a class called `TicTacToe_Minimax_Agent` that makes use of the minimax algorithm with alpha-beta pruning. In order to make the best move feasible during each turn of the game, the AI agent uses these information. It can be specified as an input parameter during agent initialization to allow for specific search depth.

My method incorporates an iterative deepening technique that systematically probes deeper levels until it either reaches the maximum permissible depth or discovers a move that ensures a win. Using this method, the AI agent may quickly find effective moves while efficiently navigating the search field.

In addition to the minimax algorithm and iterative deepening, our AI agent benefits from a heuristic evaluation function that scores game states during the search process. The heuristic function calculates a score for each player by counting the number of potential winning lines that consist of three of their symbols and one empty cell. This heuristic evaluation provides valuable guidance for the minimax algorithm, helping it make informed decisions about which moves to explore further.

Analysis of Solution:

We take two steps to determine how effectively our solution functions. First, we have the AI agent play a number of games against a random opponent using the minimax algorithm. This allows us to assess the AI agent's performance against a novice player. The AI agent will then compete in a game against a human player to demonstrate how it performs against a possibly more intelligent opponent.

The random opponent and the minimax agent play a predetermined number of games (in this case, 100). The number of times the minimax agent triumphs in these simulations is

recorded. This win rate gives us an idea of how well the AI agent plays Tic Tac Toe when up against a player without a strategy.

To make the game more interactive, my code also includes functions to print the Tic Tac Toe board and take input from a human player. This way, a human can play against the AI agent and see its capabilities firsthand.

When we run the code, it calculates the win rate of the minimax agent over the 100 games against the random opponent. This win rate serves as a measure of how effective the AI agent is in playing Tic Tac Toe.

By using the minimax algorithm with alpha-beta pruning, iterative deepening, and heuristic evaluation, the `TicTacToe_Minimax_Agent` class is able to efficiently search through the game state space and select optimal moves. This combination of techniques allows the AI agent to perform effectively against different types of opponents, from random players to intelligent human adversaries.

Test Cases

Please see the Appendix.

Extension

Tkinter GUI Improvement

To improve the user experience, I employed Tkinter to create an enhanced and interactive GUI.

Use of External tools

To reduce the number of grammatical and spelling problems, I used Grammarly to edit the work for this assignment. I also utilized ChatGPT3 to think about computational (theoretical and practical) solutions to construct an additional extension with a GUI interface and to help me determine the best ways to guarantee that the code is more effective and functional.

References

CS152 Session 8 - [4.2] Policy Search with Q-Learning (n.d.). [Lecture notes]. CS152.

Everett, K. (n.d.). How to build a Python Tic Tac Toe game. Real Python. Retrieved from <https://realpython.com/tic-tac-toe-python/>

GeeksforGeeks. (n.d.). Finding optimal move in Tic Tac Toe using minimax algorithm in game theory. Retrieved from <https://www.geeksforgeeks.org/finding-optimal-move-in-tic-tac-toe-using-minimax-algorithm-in-game-theory/>

Mathison. (n.d.). Installing Python 3.10 on Mac OS X 13. Mathison's Blog. Retrieved from <https://mathison.ch/de-ch/blog/installing-python-310-on-mac-osx-13/>

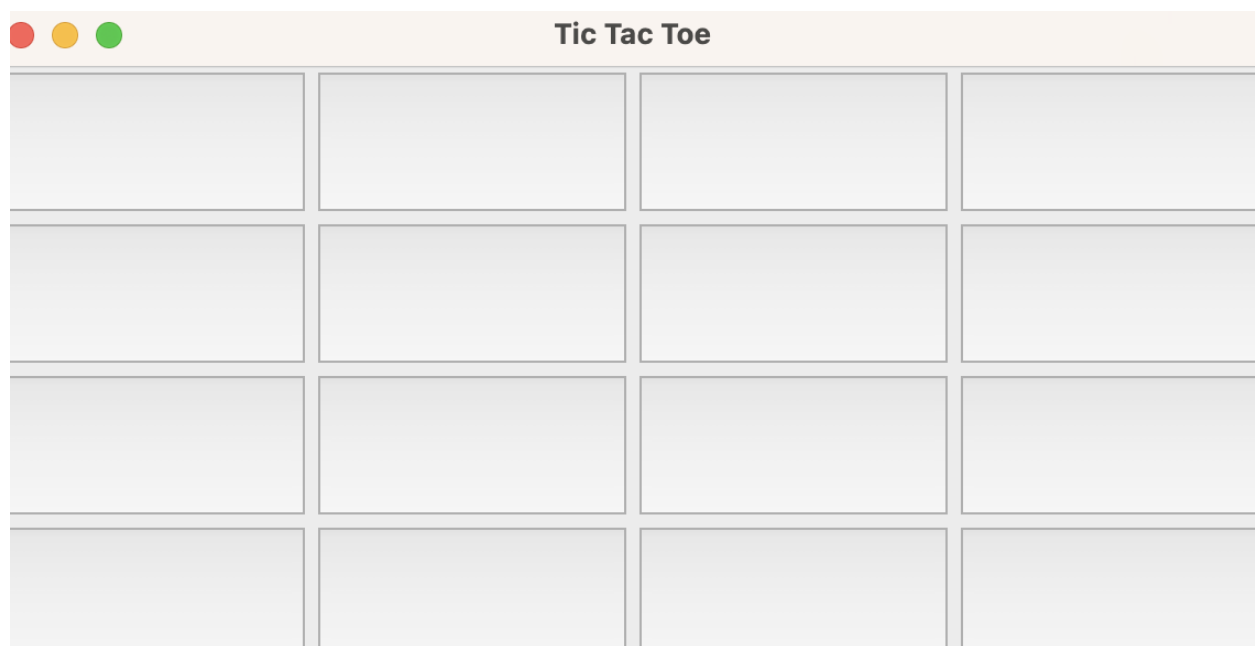
Mika, A. (n.d.). QticTacToe [Source code]. GitHub. Retrieved from <https://github.com/alirezamika/tutorials/tree/master/qtictactoe>

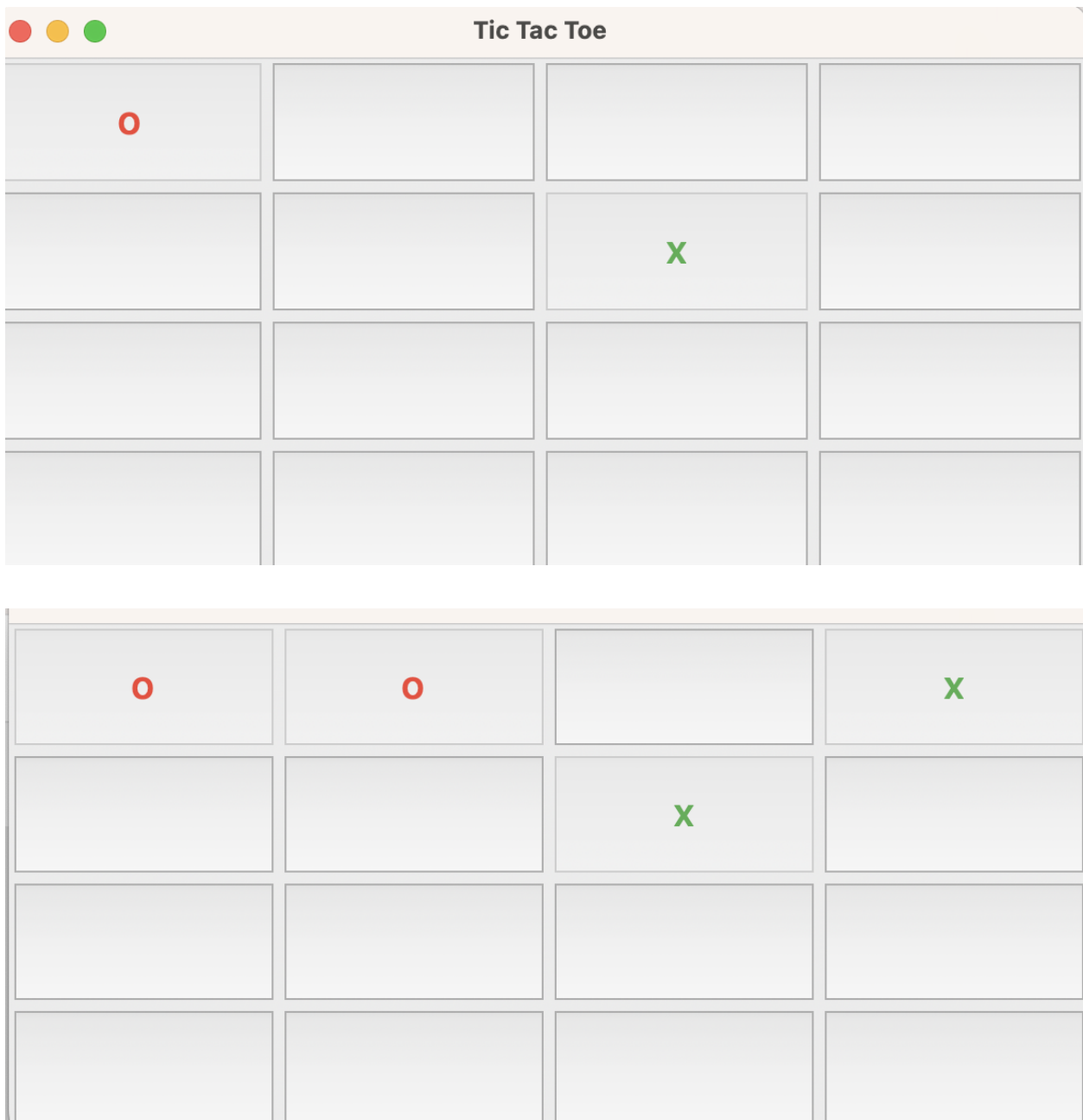
S., S. (n.d.). Mastering Tic Tac Toe with Minimax Algorithm. Gitconnected. Retried from <https://levelup.gitconnected.com/mastering-tic-tac-toe-with-minimax-algorithm-3394d65fa88f>

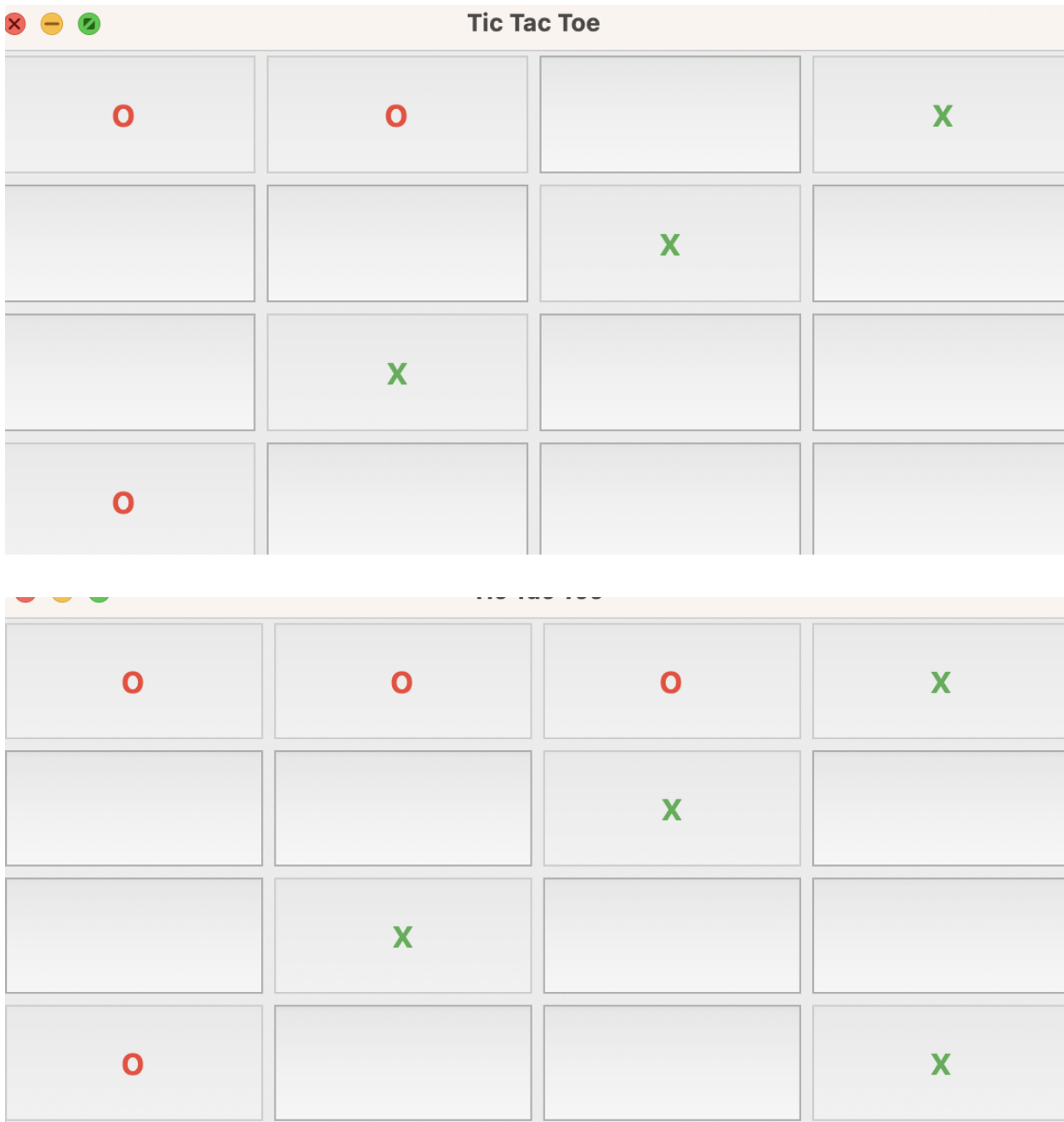
Shiffman, D. (n.d.). Coding Challenge #154: Tic Tac Toe AI with Minimax Algorithm. The Coding Train. Retried from <https://thecodingtrain.com/challenges/154-tic-tac-toe-minimax>

Appendix

Appendix A-1: Test Case 1: Human opponent is 'x'

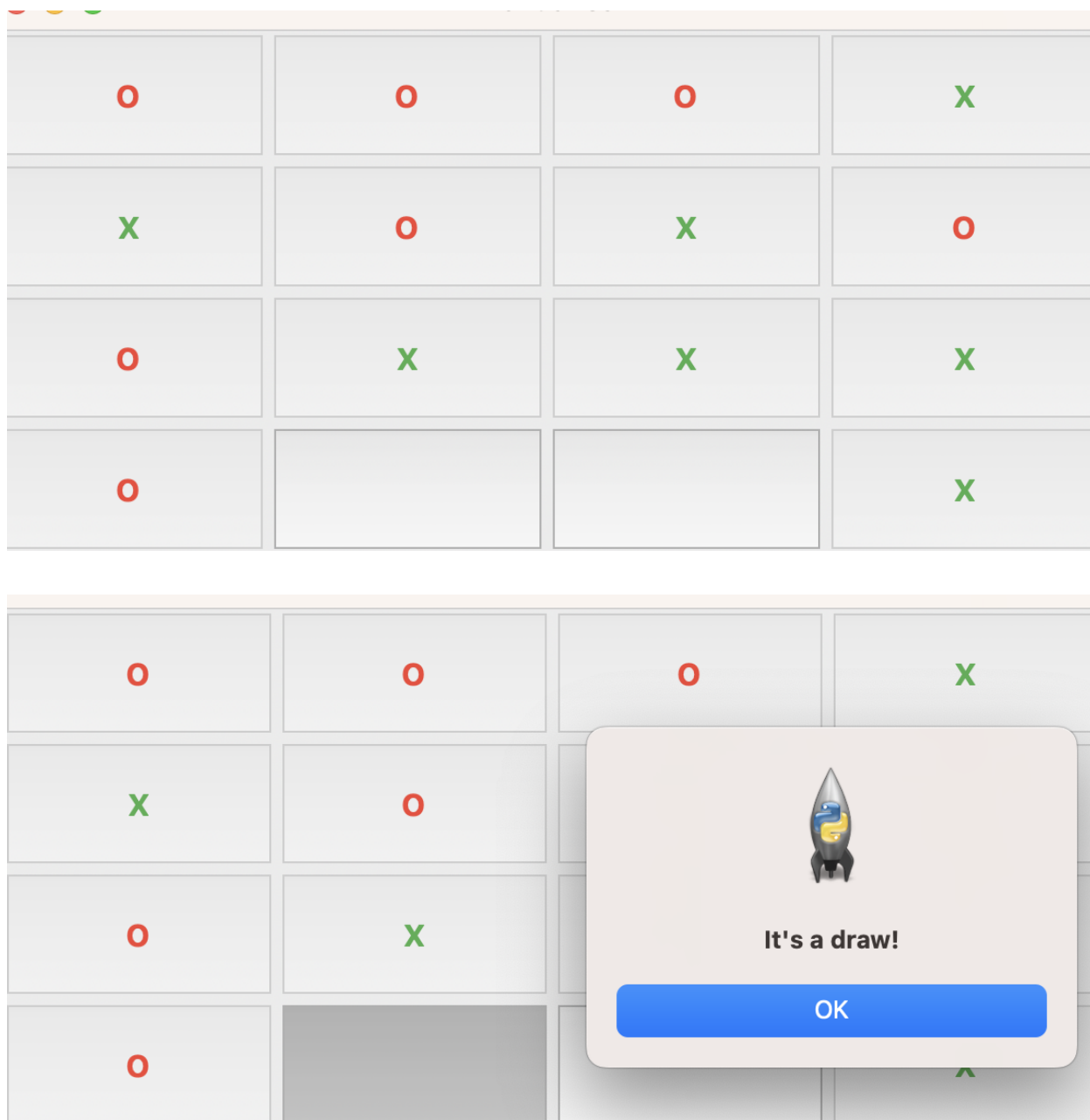




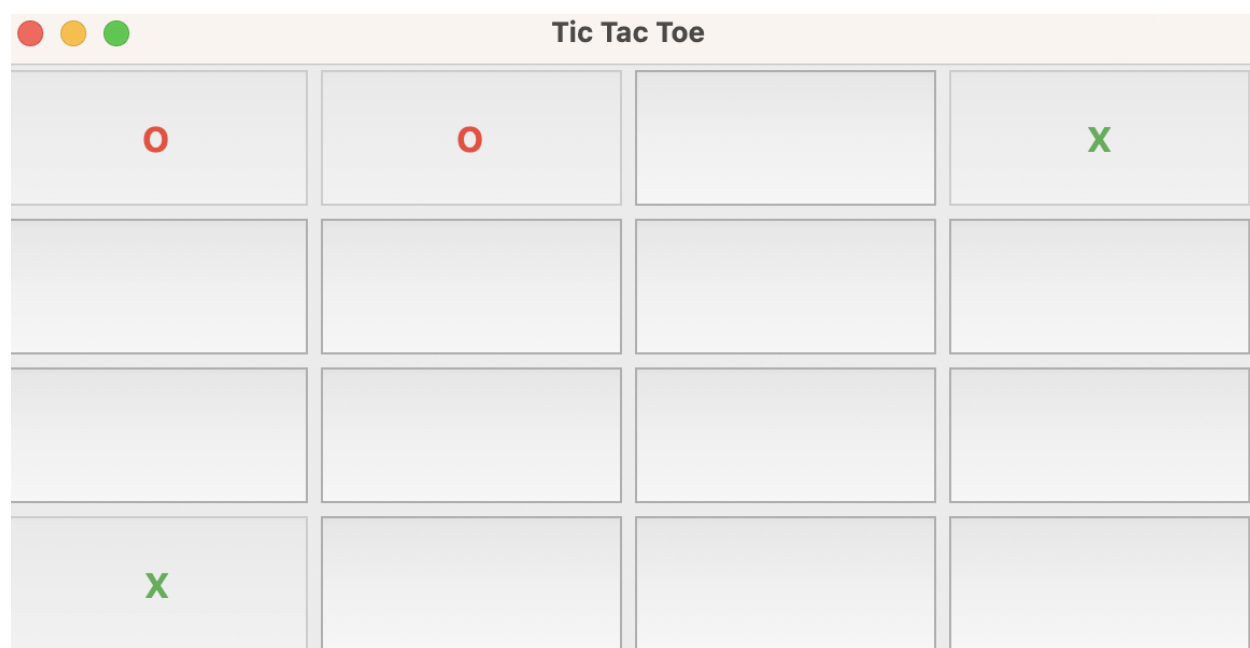
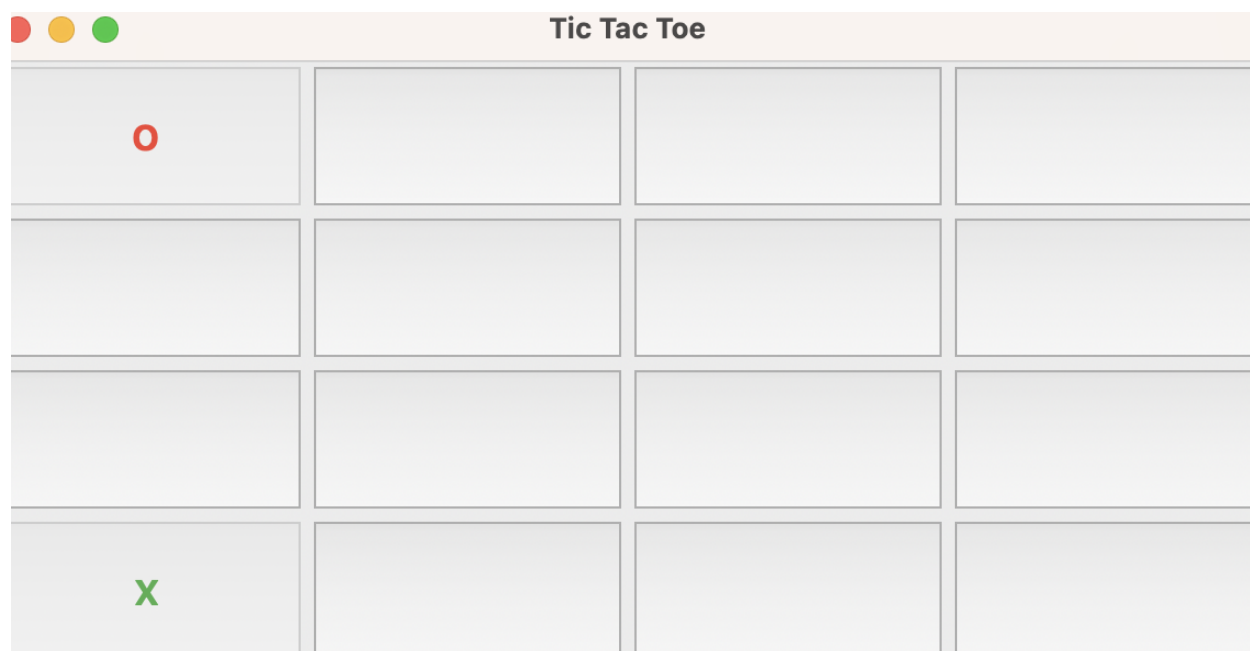


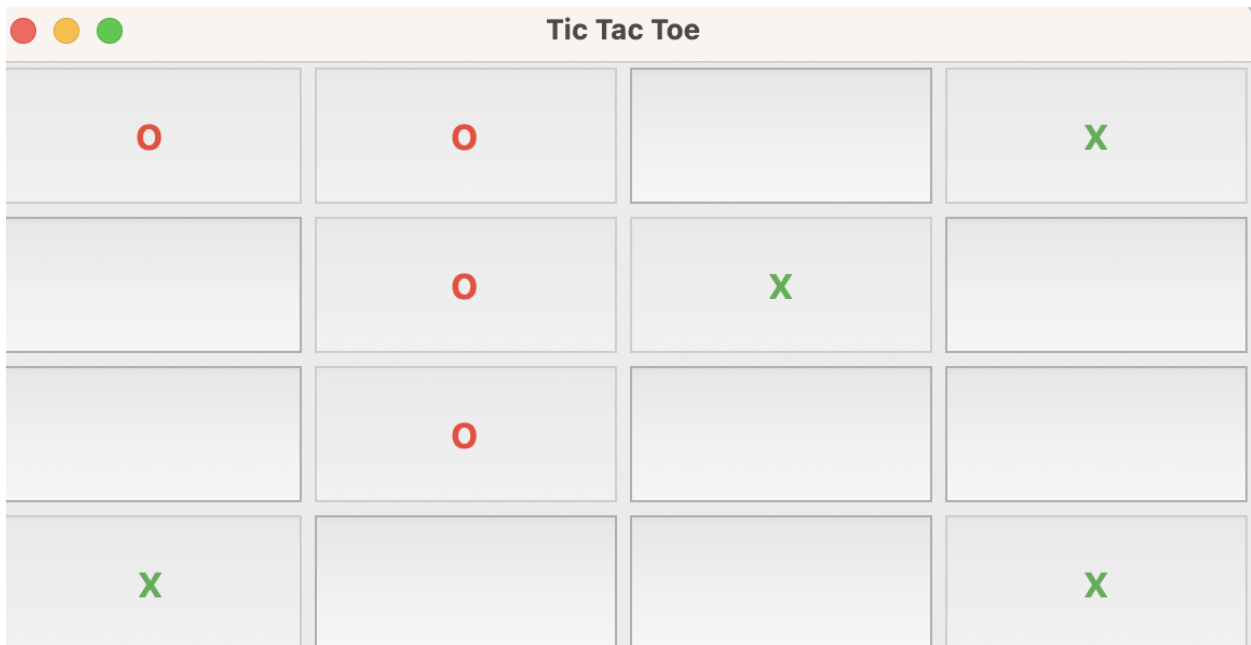
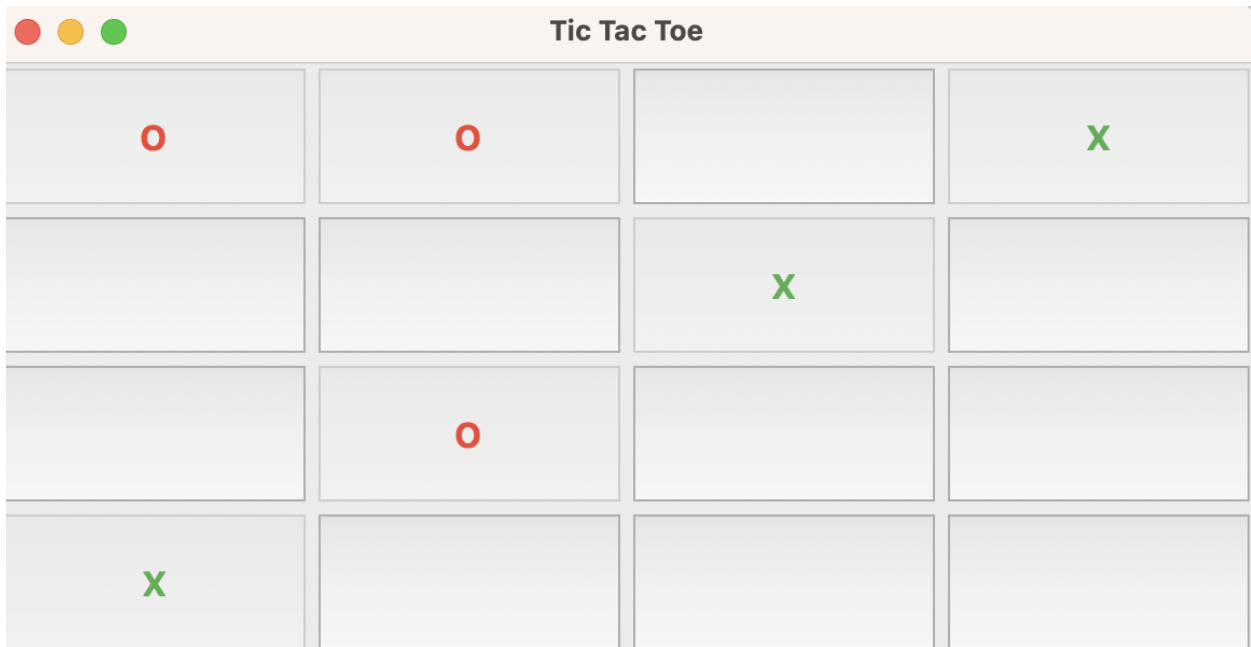
O	O	O	X
		X	O
	X		X
O			X

O	O	O	X
		X	O
O	X	X	X
O			X



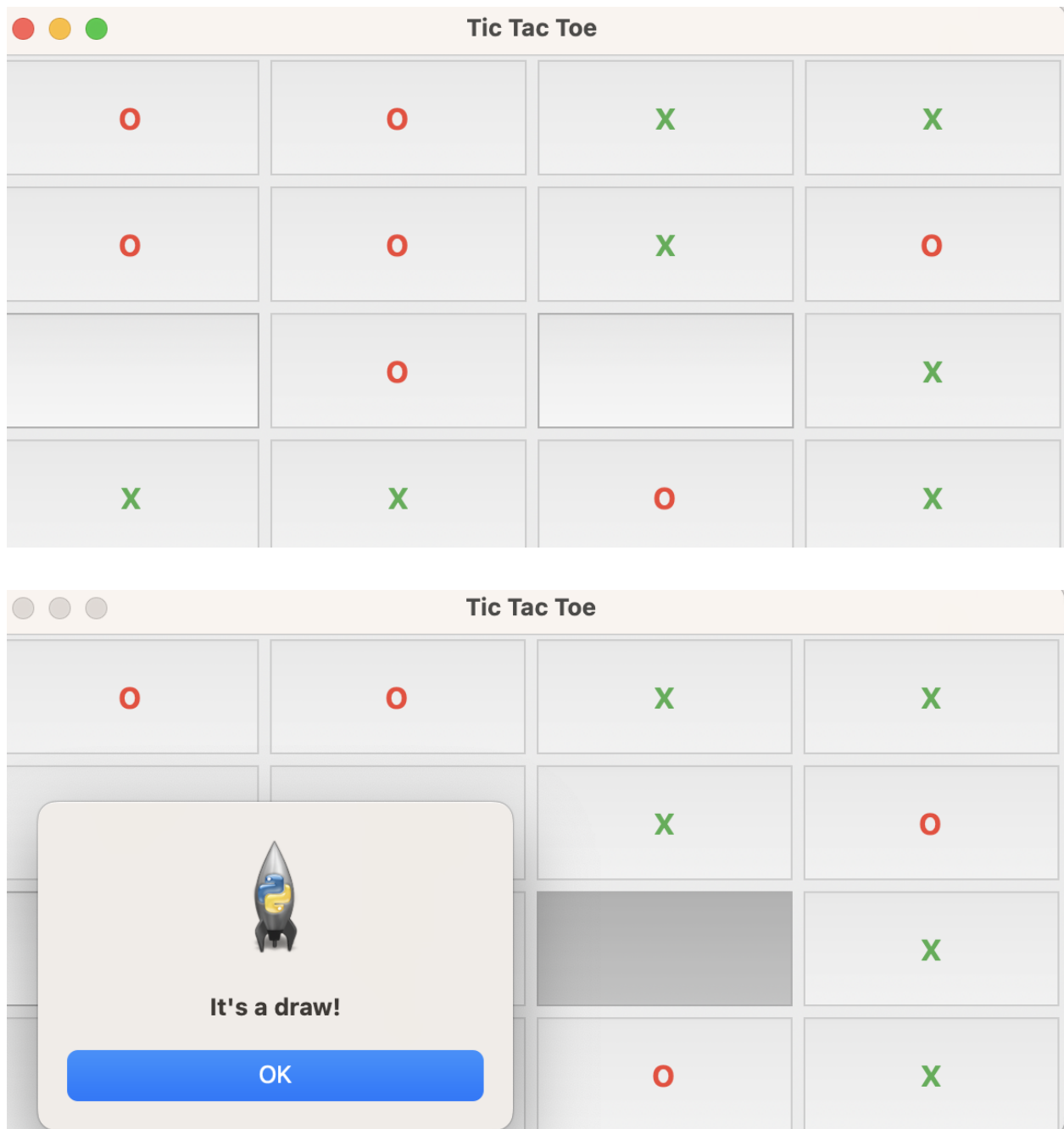
Appendix A-2: Test Case 2: Human opponent is 'x'



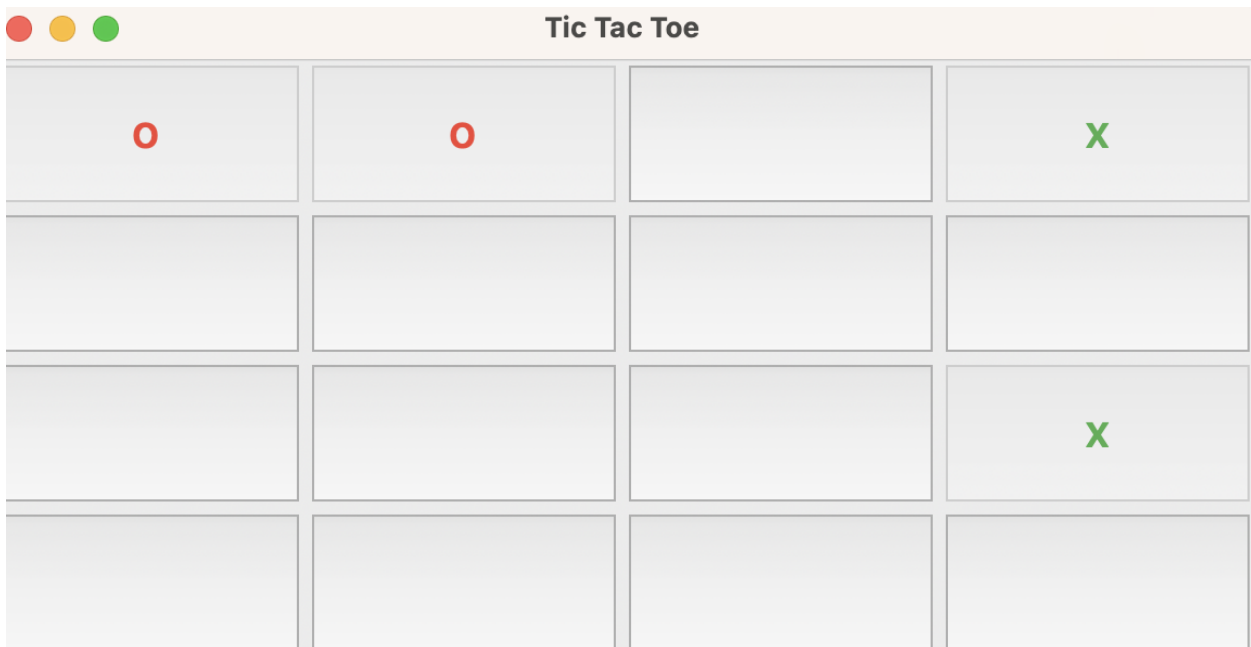


Tic Tac Toe			
O	O		X
	O	X	
	O		
X	X	O	X

Tic Tac Toe			
O	O		X
	O	X	O
	O		X
X	X	O	X



Appendix A-3: Test Case 3: Human opponent is 'x'

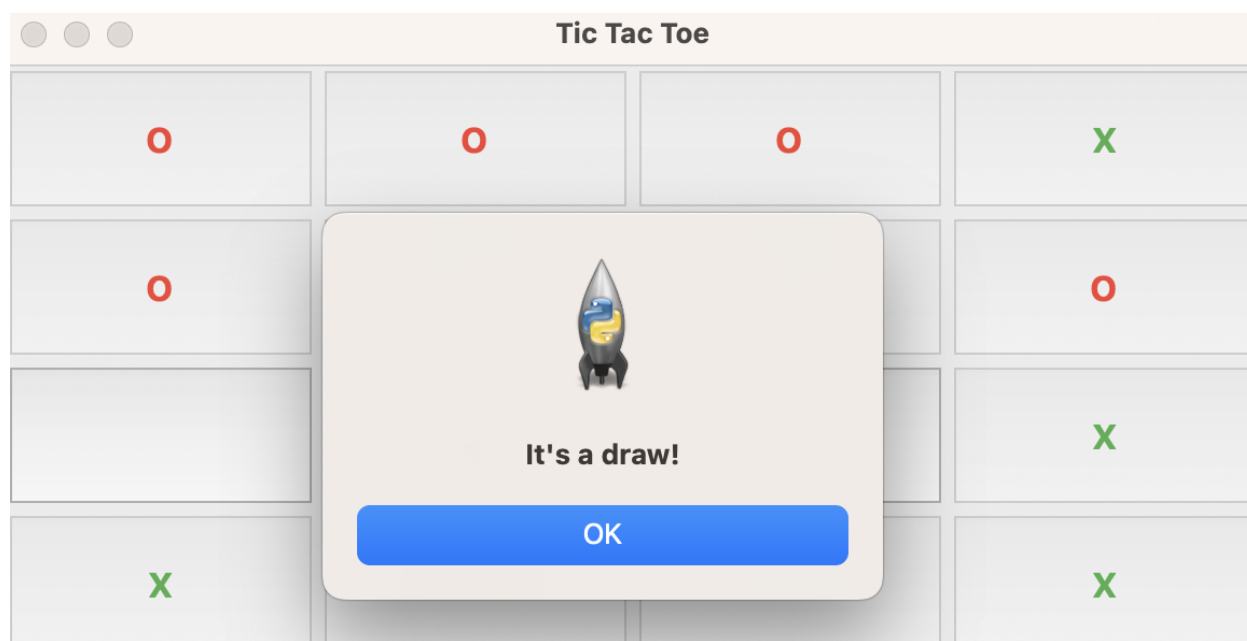


Tic Tac Toe			
O	O		X
			O
			X
			X

Tic Tac Toe			
O	O	O	X
		X	O
			X
			X

Tic Tac Toe			
O	O	O	X
		X	O
	O		X
X			X

Tic Tac Toe			
O	O	O	X
		X	O
	O		X
X	O	X	X



Appendix B-1: TicTacToe MiniMax Agent Class

```

# References:
# 1. https://github.com/alirezamika/tutorials/tree/master/qttactoe
# 2. CS152 Session 8 - [4.2] Policy Search with Q-Learning
# 3. https://realpython.com/tic-tac-toe-python/
# 4. https://mathison.ch/de-ch/blog/installing-python-310-on-mac-osx-13/
# 5. https://www.geeksforgeeks.org/finding-optimal-move-in-tic-tac-toe-using-minimax-algorithm-in-game-theory/
# 6. https://levelup.gitconnected.com/mastering-tic-tac-toe-with-minimax-algorithm-3394d65fa88f
# 7. https://thecodingtrain.com/challenges/154-tic-tac-toe-minimax

import numpy as np
import random
import copy
import matplotlib.pyplot as plt

# Define a TicTacToe agent using the minimax algorithm
class TicTacToe_Minimax_Agent:

    # Initialize the agent with a given search depth for the minimax algorithm
    def __init__(self, max_depth=4):
        self.max_depth = max_depth

    # Get a list of available actions (empty cells) in the current state
    @staticmethod
    def get_available_actions(state):
        available_actions = []
        for i in range(4):
            for j in range(4):
                if state[i][j] == 0:
                    available_actions.append((i, j))
        return available_actions

    # Check if the given player has won the game in the current state
    @staticmethod
    def check_winner(state, player):
        for row in state:
            if np.all(row == player):
                return True
        for col in state.T:
            if np.all(col == player):
                return True
        if np.all(np.diag(state) == player) or np.all(np.diag(np.fliplr(state)) == player):
            return True
        return False

    # Update the state by performing an action for the given player
    @staticmethod
    def perform_action(state, action, player):
        new_state = copy.deepcopy(state)
        new_state[action[0]][action[1]] = player
        return new_state

```

```
# Implementation of the minimax algorithm with alpha-beta pruning
def minimax(self, state, depth, maximizing_player, alpha, beta):
    if depth == 0 or self.check_winner(state, 1) or self.check_winner(state, -1) or not self.get_available_actions():
        return self.eval(state), None

    if maximizing_player:
        max_eval = -float("inf")
        best_action = None
        for action in self.get_available_actions(state):
            new_state = self.perform_action(state, action, 1)
            eval_score, _ = self.minimax(new_state, depth - 1, False, alpha, beta)
            if eval_score > max_eval:
                max_eval = eval_score
                best_action = action
            alpha = max(alpha, eval_score)
            if beta <= alpha:
                break
        return max_eval, best_action
    else:
        min_eval = float("inf")
        best_action = None
        for action in self.get_available_actions(state):
            new_state = self.perform_action(state, action, -1)
            eval_score, _ = self.minimax(new_state, depth - 1, True, alpha, beta)
            if eval_score < min_eval:
                min_eval = eval_score
                best_action = action
            beta = min(beta, eval_score)
            if beta <= alpha:
                break
        return min_eval, best_action

# Evaluation function to score the state for the minimax algorithm
def eval(self, state):
    if self.check_winner(state, 1):
        return 10
    elif self.check_winner(state, -1):
        return -10
    else:
        return self.heuristic_eval(state)
```

```

# Select the best action for the current state using the minimax algorithm
def make_move(self, state):
    best_action = None
    for depth in range(1, self.max_depth + 1):
        _, current_best_action = self.minimax(state, depth, True, -float("inf"), float("inf"))
        if current_best_action is not None:
            best_action = current_best_action

    # Stop searching if a winning move is found
    if self.eval(self.perform_action(state, best_action, 1)) == 10:
        break

    return best_action
def heuristic_eval(self, state):
    scores = [0, 0] # [score for player 1, score for player -1]

    for player in [1, -1]:
        for row in state:
            if np.sum(row == player) == 3 and np.sum(row == 0) == 1:
                scores[player == 1] += 1

        for col in state.T:
            if np.sum(col == player) == 3 and np.sum(col == 0) == 1:
                scores[player == 1] += 1

        for diag in [np.diag(state), np.diag(np.fliplr(state))]:
            if np.sum(diag == player) == 3 and np.sum(diag == 0) == 1:
                scores[player == 1] += 1

    return scores[0] - scores[1]

```

Appendix B-2: Random player Class & simulation of games between the Random player and Ai agent

```

# Define a random opponent for the TicTacToe game
class RandomOpponent:

    # Get a list of available actions (empty cells) in the current state
    @staticmethod
    def get_available_actions(state):
        available_actions = []
        for i in range(4):
            for j in range(4):
                if state[i][j] == 0:
                    available_actions.append((i, j))
        return available_actions

    # Select a random action from the list of available actions
    def make_move(self, state):
        actions = self.get_available_actions(state)
        return random.choice(actions)

```

```

# Simulate multiple games between the minimax agent and the random opponent
def simulate_games(num_games, minimax_agent, minimax_depth=4):
    random_opponent = RandomOpponent()

    minimax_wins = 0
    random_wins = 0
    draws = 0

    # Play a specified number of games
    for _ in range(num_games):
        state = np.zeros((4, 4), dtype=int)
        current_player = 1

        # Continue playing until the game is over
        while True:
            if current_player == 1:
                action = minimax_agent.make_move(state)
            else:
                action = random_opponent.make_move(state)

            state = TicTacToe_Minimax_Agent.perform_action(state, action, current_player)

            if TicTacToe_Minimax_Agent.check_winner(state, current_player):
                if current_player == 1:
                    minimax_wins += 1
                else:
                    random_wins += 1
                break

            if not TicTacToe_Minimax_Agent.get_available_actions(state):
                draws += 1
                break

            current_player *= -1

        # Return the results of the simulated games
    return {
        "minimax_wins": minimax_wins,
        "random_wins": random_wins,
        "draws": draws,
    }

# Calculate the minimax agent's win rate
num_games = 100
results = simulate_games(num_games, TicTacToe_Minimax_Agent(), minimax_depth=4)
minimax_win_rate = results["minimax_wins"] / num_games
print(f"Minimax agent win rate over {num_games} games: {minimax_win_rate:.2%}")

# Print the TicTacToe board with human-readable symbols

```

Appendix B-3: The Rest of the code that handles printing the board and letting User play a game with the AI opponent

```

# Print the TicTacToe board with human-readable symbols
def print_board(board):
    symbols = {0: '.', 1: 'X', -1: 'O'}
    for row in board:
        print(' '.join([symbols[cell] for cell in row]))
    print()

# Prompt the human player to make a move
def human_move(state):
    while True:
        try:
            move = input("Enter your move (row, col): ")
            row, col = map(int, move.split(','))
            if state[row][col] == 0:
                return (row, col)
            else:
                print("Invalid move. Cell is already filled. Try again.")
        except ValueError:
            print("Invalid input. Please enter row and col as integers separated by a comma.")
        except IndexError:
            print("Invalid input. Please enter row and col values within the range of 0-3.")

# Play a TicTacToe game between a human player and the AI agent
def play_game(ai_agent):
    state = np.zeros((4, 4), dtype=int)
    current_player = 1 # Human player
    human_symbol, ai_symbol = 1, -1

    # Continue playing until the game is over
    while True:
        # Print the current state of the board
        print_board(state)

        # Let the human player or the AI agent make a move
        if current_player == human_symbol:
            action = human_move(state)
        else:
            action = ai_agent.make_move(state)

        # Update the state of the board with the new move
        state = TicTacToe_Minimax_Agent.perform_action(state, action, current_player)

        # Check if there's a winner
        if TicTacToe_Minimax_Agent.check_winner(state, current_player):
            if current_player == human_symbol:
                print("Congratulations! You won!")
            else:
                print("The AI won.")
            print_board(state)
            break

        # Check if the game is a draw (no more available actions)
        if not TicTacToe_Minimax_Agent.get_available_actions(state):
            print("It's a draw!")
            print_board(state)
            break

        # Switch the current player
        current_player *= -1

    # Create the AI agent and start the game
    ai_agent = TicTacToe_Minimax_Agent()
    print("Tic Tac Toe - You are playing as 'X', the AI is playing as 'O'")
    play_game(ai_agent)

```

Appendix B-4: Output

```

Minimax agent win rate over 100 games: 89.00%
Tic Tac Toe - You are playing as 'X', the AI is playing as 'O'
. . . .
. . . .
. . . .
. . . .

Enter your move (row, col): 0,0
X . . .
. . . .
. . . .
. . . .

X O . .
. . . .
. . . .
. . . .

Enter your move (row, col): 3,0
X O . .
. . . .
. . . .
X . . .

X O O .
. . . .
. . . .
X . . .

Enter your move (row, col): 3,3
X O O .
. . . .
. . . .
X . . X

X O O O
. . . .
. . . .
X . . X

Enter your move (row, col): 2,2
X O O O
. . . .
. . X .
X . . X

X O O O
O . . .
. . X .
X . . X

```

Appendix C-1: Extended version of the implementation with Tkinter.

```
import numpy as np
import random
import copy
import tkinter as tk
from tkinter import messagebox
from tkinter import font

# Define a TicTacToe agent using the minimax algorithm
class TicTacToe_Minimax_Agent:

    # Initialize the agent with a given search depth for the minimax algorithm
    def __init__(self, max_depth=4):
        self.max_depth = max_depth

    # Get a list of available actions (empty cells) in the current state
    @staticmethod
    def get_available_actions(state):
        available_actions = []
        for i in range(4):
            for j in range(4):
                if state[i][j] == 0:
                    available_actions.append((i, j))
        return available_actions

    # Check if the given player has won the game in the current state
    @staticmethod
    def check_winner(state, player):
        for row in state:
            if np.all(row == player):
                return True
        for col in state.T:
            if np.all(col == player):
                return True
        if np.all(np.diag(state) == player) or np.all(np.diag(np.fliplr(state)) == player):
            return True
        return False
```

```
# Update the state by performing an action for the given player
@staticmethod
def perform_action(state, action, player):
    new_state = copy.deepcopy(state)
    new_state[action[0]][action[1]] = player
    return new_state

# Select the best action for the current state using the minimax algorithm
def make_move(self, state):
    best_action = None
    for depth in range(1, self.max_depth + 1):
        _, current_best_action = self.minimax(state, depth, True, -float("inf"), float("inf"))
        if current_best_action is not None:
            best_action = current_best_action

        # Stop searching if a winning move is found
        if self.eval(self.perform_action(state, best_action, 1)) == 10:
            break

    return best_action
```

```
def heuristic_eval(self, state):
    scores = [0, 0] # [score for player 1, score for player -1]

    for player in [1, -1]:
        for row in state:
            if np.sum(row == player) == 3 and np.sum(row == 0) == 1:
                scores[player == 1] += 1

        for col in state.T:
            if np.sum(col == player) == 3 and np.sum(col == 0) == 1:
                scores[player == 1] += 1

        for diag in [np.diag(state), np.diag(np.fliplr(state))]:
            if np.sum(diag == player) == 3 and np.sum(diag == 0) == 1:
                scores[player == 1] += 1

    return scores[0] - scores[1]
```

```
# Implementation of the minimax algorithm with alpha-beta pruning
def minimax(self, state, depth, maximizing_player, alpha, beta):
    if depth == 0 or self.check_winner(state, 1) or self.check_winner(state, -1) or not self.get_available_actions(state):
        return self.eval(state), None

    if maximizing_player:
        max_eval = -float("inf")
        best_action = None
        for action in self.get_available_actions(state):
            new_state = self.perform_action(state, action, 1)
            eval_score, _ = self.minimax(new_state, depth - 1, False, alpha, beta)
            if eval_score > max_eval:
                max_eval = eval_score
                best_action = action
            alpha = max(alpha, eval_score)
            if beta <= alpha:
                break
        return max_eval, best_action
    else:
        min_eval = float("inf")
        best_action = None
        for action in self.get_available_actions(state):
            new_state = self.perform_action(state, action, -1)
            eval_score, _ = self.minimax(new_state, depth - 1, True, alpha, beta)
            if eval_score < min_eval:
                min_eval = eval_score
                best_action = action
            beta = min(beta, eval_score)
            if beta <= alpha:
                break
        return min_eval, best_action
```

```
# Evaluation function to score the state for the minimax algorithm
def eval(self, state):
    if self.check_winner(state, 1):
        return 10
    elif self.check_winner(state, -1):
        return -10
    else:
        return self.heuristic_eval(state)

class RandomOpponent:
    @staticmethod
    def get_available_actions(state):
        available_actions = []
        for i in range(4):
            for j in range(4):
                if state[i][j] == 0:
                    available_actions.append((i, j))
        return available_actions

    def make_move(self, state):
        actions = self.get_available_actions(state)
        return random.choice(actions)
```

```
def simulate_games(num_games, minimax_agent, minimax_depth=4):
    random_opponent = RandomOpponent()

    minimax_wins = 0
    random_wins = 0
    draws = 0

    for _ in range(num_games):
        state = np.zeros((4, 4), dtype=int)
        current_player = 1

        while True:
            if current_player == 1:
                action = minimax_agent.make_move(state)
            else:
                action = random_opponent.make_move(state)

            state = TicTacToe_Minimax_Agent.perform_action(state, action, current_player)

            if TicTacToe_Minimax_Agent.check_winner(state, current_player):
                if current_player == 1:
                    minimax_wins += 1
                else:
                    random_wins += 1
                break

            if not TicTacToe_Minimax_Agent.get_available_actions(state):
                draws += 1
                break

            current_player *= -1

    return {
        "minimax_wins": minimax_wins,
        "random_wins": random_wins,
        "draws": draws,
    }
```

```
num_games = 100
results = simulate_games(num_games, TicTacToe_Minimax_Agent(), minimax_depth=4)
minimax_win_rate = results["minimax_wins"] / num_games
print(f"Minimax agent win rate over {num_games} games: {minimax_win_rate:.2%}")

def print_board(board):
    symbols = {0: '.', 1: 'X', -1: 'O'}
    for row in board:
        print(' '.join([symbols[cell] for cell in row]))
    print()

def human_move(state):
    while True:
        try:
            move = input("Enter your move (row, col): ")
            row, col = map(int, move.split(','))
            if state[row][col] == 0:
                return (row, col)
            else:
                print("Invalid move. Cell is already filled. Try again.")
        except ValueError:
            print("Invalid input. Please enter row and col as integers separated by a comma.")
        except IndexError:
            print("Invalid input. Please enter row and col values within the range of 0-3.")
```

```
def play_game(ai_agent):
    state = np.zeros((4, 4), dtype=int)
    current_player = 1 # Human player
    human_symbol, ai_symbol = 1, -1

    while True:
        print_board(state)

        if current_player == human_symbol:
            action = human_move(state)
        else:
            action = ai_agent.make_move(state)

        state = TicTacToe_Minimax_Agent.perform_action(state, action, current_player)

        if TicTacToe_Minimax_Agent.check_winner(state, current_player):
            if current_player == human_symbol:
                print("Congratulations! You won!")
            else:
                print("The AI won.")
            print_board(state)
            break

        if not TicTacToe_Minimax_Agent.get_available_actions(state):
            print("It's a draw!")
            print_board(state)
            break

        current_player *= -1
```

```
# ai_agent = TicTacToe_Minimax_Agent()
# print("Tic Tac Toe – You are playing as 'X', the AI is playing as 'O'")
# play_game(ai_agent)
```

```
class TicTacToeGUI:
    def __init__(self, ai_agent):
        self.ai_agent = ai_agent
        self.window = tk.Tk()
        self.window.title("Tic Tac Toe")
        self.state = np.zeros((4, 4), dtype=int)
        self.current_player = 1
        self.buttons = [[None for _ in range(4)] for _ in range(4)]
        self.create_widgets()
        self.window.mainloop()

    def create_widgets(self):
        custom_font = font.Font(size=16, weight='bold')
        for i in range(4):
            for j in range(4):
                self.buttons[i][j] = tk.Button(self.window, text="", width=10, height=3,
                                                command=lambda row=i, col=j: self.human_move(row, col))
                self.buttons[i][j].config(font=custom_font)
                self.buttons[i][j].grid(row=i, column=j)

    def human_move(self, row, col):
        if self.state[row][col] == 0:
            self.state[row][col] = self.current_player
            self.update_button_text(row, col)
            if TicTacToe_Minimax_Agent.check_winner(self.state, self.current_player):
                self.game_over(f"Player {self.current_player} wins!")
                return
            if not TicTacToe_Minimax_Agent.get_available_actions(self.state):
                self.game_over("It's a draw!")
                return
            self.current_player *= -1
            self.ai_move()
        else:
            messagebox.showerror("Invalid Move", "This cell is already filled. Please select another one.")
```

```
def ai_move(self):
    action = self.ai_agent.make_move(self.state)
    self.state[action[0]][action[1]] = self.current_player
    self.update_button_text(action[0], action[1])
    if TicTacToe_Minimax_Agent.check_winner(self.state, self.current_player):
        self.game_over(f"Player {self.current_player} wins!")
        return
    if not TicTacToe_Minimax_Agent.get_available_actions(self.state):
        self.game_over("It's a draw!")
        return
    self.current_player *= -1

def update_button_text(self, row, col):
    text = "X" if self.state[row][col] == 1 else "O"
    color = "#4CAF50" if self.state[row][col] == 1 else "#F44336"
    self.buttons[row][col].config(text=text, state=tk.DISABLED, disabledforeground=color)

def game_over(self, message):
    messagebox.showinfo("Game Over", message)
    self.window.destroy()

if __name__ == "__main__":
    ai_agent = TicTacToe_Minimax_Agent()
    print("Tic Tac Toe - You are playing as 'X', the AI is playing as 'O'")
    app = TicTacToeGUI(ai_agent)
```

Appendix C-2: Output of Extended version of the implementation with Tkinter.

